

Mathematical / Carry Pattern

LeetCode 2 – Add Two Numbers ★★★★★

Concept

- Traverse both lists simultaneously.
- Maintain a **carry** from the previous digit addition.
- Create a new node with `(sum % 10)`.
- Update `carry = sum / 10`.
- Continue until both lists and carry are exhausted.

Time: `O(max(n, m))`

Space: `O(max(n, m))` (output list)

Edge Cases

- Lists have different lengths.
- Final carry remains (e.g., `999 + 1`).
- One list is empty.
- Sum is `0`.

Concept Code Snippet

```
ListNode dummy(-1);
ListNode* tail = &dummy;

int carry = 0;

while (l1 || l2 || carry) {
    int sum = carry;

    if (l1) {
        sum += l1->val;
        l1 = l1->next;
    }

    if (l2) {
        sum += l2->val;
        l2 = l2->next;
    }

    carry = sum / 10;
    tail->next = new ListNode(sum % 10);
    tail = tail->next;
}

return dummy.next;
```

Pattern to Remember

Traverse Both Lists → Add Digits + Carry → Create Node → Update Carry → Repeat Until Carry Ends

LeetCode 445 – Add Two Numbers II ★★★★★☆

Concept

- Digits are stored in **forward order**, so we cannot add directly.
- Push digits of both lists onto **stacks**.
- Pop from both stacks (least significant digits first), add with carry.
- Insert each new node at the **front** of the answer list.

Time: `O(n + m)`

Space: `O(n + m)` (stacks)

*******Alternative:** Reverse both lists → solve like LC 2 → reverse result (or restore lists if required).

Edge Cases

- Different length lists.
- Final carry remains.
- One list is empty.
- Input like `999 + 1`.

Concept Code Snippet

```
stack<int> s1, s2;

while (l1) {
    s1.push(l1->val);
    l1 = l1->next;
}

while (l2) {
    s2.push(l2->val);
    l2 = l2->next;
}

int carry = 0;
ListNode* head = nullptr;

while (!s1.empty() || !s2.empty() || carry) {
    int sum = carry;

    if (!s1.empty()) {
        sum += s1.top();
        s1.pop();
    }

    if (!s2.empty()) {
        sum += s2.top();
        s2.pop();
    }

    carry = sum / 10;

    ListNode* node = new ListNode(sum % 10);
    node->next = head;
    head = node;
}

return head;
```

Pattern to Remember

Push to Stacks → Pop + Add + Carry → Insert at Front → Repeat Until Carry Ends

LeetCode 1290 – Convert Binary Number in a Linked List to Integer ★★☆☆

Concept

- Traverse the linked list from left to right.
- Treat the current answer as a binary number:
 - Left shift by 1 (`ans << 1`) to make room for the next bit.
 - Add the current node's value.

Time: `O(n)`

Space: `O(1)`

Edge Cases

- Single node (`0` or `1`).
- All zeros.
- All ones.
- Leading zeros.

Concept Code Snippet

```
int ans = 0;

while (head) {
    ans = (ans << 1) | head->val;
    head = head->next;
}

return ans;
```

Equivalent:

```
ans = ans * 2 + head->val;
```

Pattern to Remember

Traverse → Left Shift Answer → Add Current Bit

LeetCode 2816 – Double a Number Represented as a Linked List ★★★★★☆

Concept

Method 1 (Most Common): Reverse + Carry

- Reverse the list.
- Traverse and multiply each digit by 2.
- Handle carry.
- Add a new node if carry remains.
- Reverse back.

Time: O(n)

Space: O(1)

Alternative (No Reverse): Use recursion to propagate carry from the end.

Edge Cases

- Final carry (e.g., 999 → 1998).
- Single node.
- Leading carry creates a new head.
- Number is 0.

Concept Code Snippet

```
// Reverse list first

int carry = 0;
ListNode* curr = head;
ListNode* prev = nullptr;

while (curr) {
    int val = curr->val * 2 + carry;
    curr->val = val % 10;
    carry = val / 10;

    prev = curr;
    curr = curr->next;
}

if (carry)
    prev->next = new ListNode(carry);

// Reverse again
```

Pattern to Remember

Reverse → Multiply by 2 + Carry → Add Final Carry → Reverse Back